

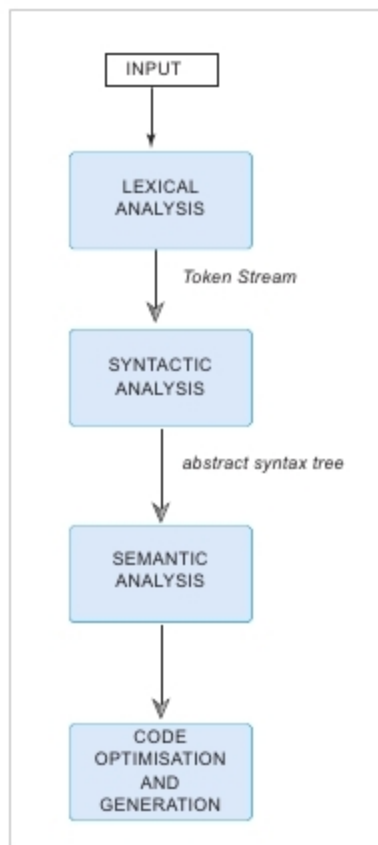


Figure 2: Stages involved in parsing a file

which builds the AST. An AST is a hierarchical tree-like data structure that represents the syntax of the program. The stages involved in parsing can be viewed in Figure 2.

How to list all the classes in the input package

The Java parser first parses through each and every file in the input Java package. The next step is to filter the Java files from the list of files. From each of the Java files, we get the tokens and then build the AST.

In the AST, an adapter called the *VoidVisitorAdapter* is used to list all classes in a file. To learn how to list the

inner classes in a file, go to <https://tomassetti.me/getting-started-with-javaparser-analyzing-java-code-programmatically/>.

A code snippet for listing the classes is given below.

```

1. private void listClasses(File projectDir) {
2.   new DirExplorer((level, path, file) -> path.endsWith(".java"), (level, path, file) -> {
3.     try {
4.       new VoidVisitorAdapter<Object>() {
5.         @Override
6.         public void visit(ClassOrInterfaceDeclaration n, Object arg) {
7.           super.visit(n, arg);
8.           s.add(n.getName());
9.           try{
10.            Class c = Class.forName(n.getName());
11.            System.out.println(c.getSuperclass());
12.          }
13.          catch (Exception e){
14.          }
15.        }
16.      }.visit(JavaParser.parse(file), null);
17.    } catch (ParseException | IOException e) {
18.      new RuntimeException(e);
19.    }
20.  }).explore(projectDir);
21. }

```

How to list the inner and outer classes in the input package

When given the package as input, the Java parser parses through each file. It builds the AST for each file, which is a *CompilationUnit*, and this represents all the contents

of the file. The code:

```
CompilationUnit cu = JavaParser.parse(file_name)
```

...builds the AST for a file. This code is available as an open source project. The code of the input package can be found at the GitHub link <https://github.com/javaparser/javasymbolsolver>. The code snippet is given below.

```

1. void Find_InnerClasses(String Directory){
2.   File projectDir = new File(Directory);
3.   copyArrayList(ListFiles.listClasses(projectDir));
4.   CompilationUnit cu;
5.   for(int i =0; i< files_list.size(); i++){
6.     try (FileInputStream in = new FileInputStream(files_list.get(i))) {
7.       cu = JavaParser.parse(in);
8.       List<TypeDeclaration> types = cu.getTypes();
9.       ArrayList<String> parents = new ArrayList<>();
10.      for (TypeDeclaration typeDeclaration : types) {
11.        printTypes(typeDeclaration, parents);
12.      }
13.    }
14.    catch(Exception e){
15.      System.out.println(e);
16.    }
17.    new Build_InnerClassesTree(s).TreeExample();
18.    s.clear();
19.  }
20. }

```

How to list the inheritance relations in the package

To list the inheritance relations between the classes in a given Java package, parse through each and every file in the package. By the end of this step, you will get the list of all the words in that file. Then search for the *extends* keyword (which is the keyword for inheritance). Repeat this process for all the files in a given package. The above results are stored in the form of a *HashMap*, where the key represents a parent class or interface (or even an abstract class) and the value represents either a class, abstract classes or interfaces that inherit the parent class.

This tool supports single inheritance, multi-level inheritance and also hierarchical inheritance.

The code for listing the inheritance relation among classes is given below:

```

1. public void getHashMap(){
2.   int flag =0;
3.   for(int i=0 ; i<list.size();i++){
4.     if((list.get(i).equals("extends")) && i >= 2) {
5.       if(!find_keyword(list.get(i-1))) && (find_keywords(list.get(i-2))) && (i <=

```